Comparison of Lightweight Encryption Algorithms

Progress Report - Phase 1

Rajesh Kumar Jena (B122088) Soumil Kumar (B122114) Punit Kumar (B422041)

International Institute of Information Technology
Bhubaneswar
Supervisor: Prof. Debasish Jena

March 2026

Outline

1. Introduction
2. Project Status
3. Implementation
4. Results
5. Conclusion

Motivation

- IoT deployments place cryptography on devices with limited CPU, memory, and energy
- Conventional algorithms are often too expensive for such platforms
- *Lightweight cryptography* targets security with lower computational and memory cost

Problem Statement and Objectives

Problem Statement

- AES, RSA, and ECC often exceed microcontroller resource limits
- Lightweight cryptography addresses this gap without sacrificing core security goals

Objectives

- Implementation of lightweight ciphers (PRESENT, SPECK, ASCON) in Rust
- Benchmarking on Raspberry Pi Pico and host machine
- Performance analysis and comparison
- Security considerations for embedded systems

Selected Algorithms

- **PRESENT**: Hardware-oriented 64-bit SPN cipher with 80/128-bit keys
- **SPECK**: Software-optimized ARX cipher (NSA-designed)
- **ASCON**: NIST Lightweight Cryptography Standard (2023) - authenticated encryption

Current Progress

Task	Status	Notes
Literature Survey	Complete	100%
Algorithm Implementation	Complete	Core ciphers done
Test Vector Verification	Partial	Basic tests pass, KAT pending
Host Benchmarks	Partial	Initial metrics collected
Pico Hardware Benchmarks	Partial	Initial benchmarks run
ROM/RAM Analysis	Partial	Host analysis done

Completed:

- Rust implementations: PRESENT, SPECK, and ASCON
- Mode implementations: ECB, CBC, CTR for PRESENT and SPECK
- Embedded `no_std` library for target hardware
- Basic validation: test vectors, consistency checks, ASCON tag verification

Pending:

- NIST KAT tests, ROM/RAM analysis on Pico

PRESENT

- 64-bit SPN block cipher
- 80-bit and 128-bit keys
- Designed for ultra-lightweight hardware
- Round steps: AddRoundKey, S-box, permutation

```
const SBOX: [u8; 16] = [
    0xC, 0x5, 0x6, 0xB, 0x9, 0x0, 0xA, 0xD,
    0x3, 0xE, 0xF, 0x8, 0x4, 0x7, 0x1, 0x2,
];

fn sbox_layer(state: u64) -> u64 {
    let mut result = 0u64;
    for i in 0..16 {
        let nibble = ((state >> (i * 4)) & 0xF) as usize;
        result |= u64::from(SBOX[nibble]) << (i * 4);
    }
    result
}
```

SPECK

- Software-optimized ARX cipher
- Feistel-like word-based structure
- Efficient on microcontrollers

```rust
pub enum SpeckVariant {
    Speck32_64,
    Speck48_72,
    Speck48_96,
    Speck64_96,
    Speck64_128,
}

impl SpeckVariant {
    pub const fn rounds(self) -> usize {
        match self {
            Self::Speck64_96 => 26,
            Self::Speck64_128 => 27,
            _ => 22,
        }
    }
}
```

ASCON

- NIST Lightweight Cryptography Standard (2023)
- Sponge-duplex construction
- 320-bit internal state
- Supports authenticated encryption and hashing

```
const ROUNDS_A: usize = 12;
const ROUNDS_B: usize = 6;

const fn p_sbox(s0: u64, s1: u64, s2: u64, s3: u64, s4: u64)
    -> (u64, u64, u64, u64, u64) {
    let s0 = s0 ^ s4;
    let s4 = s4 ^ s3;
    let s2 = s2 ^ s1;
    let t0 = (!s0) & s1;
    let t1 = (!s1) & s2;
    (s0, s1, s2, s3, s4)
}
```

Methodology

Evaluation Criteria

- **Execution Time**: ns per block (measured via hardware timer)
- **Cycles Per Byte (CPB)**: Platform-independent efficiency metric
- **Throughput**: Mbps for comparing speed
- **Memory**: ROM/RAM footprint analysis

Hardware Platform

- Raspberry Pi Pico (ARM Cortex-M0+, 133 MHz, 264 KB SRAM)
- Rust `no_std` embedded implementation
- Release mode with LTO and size optimization

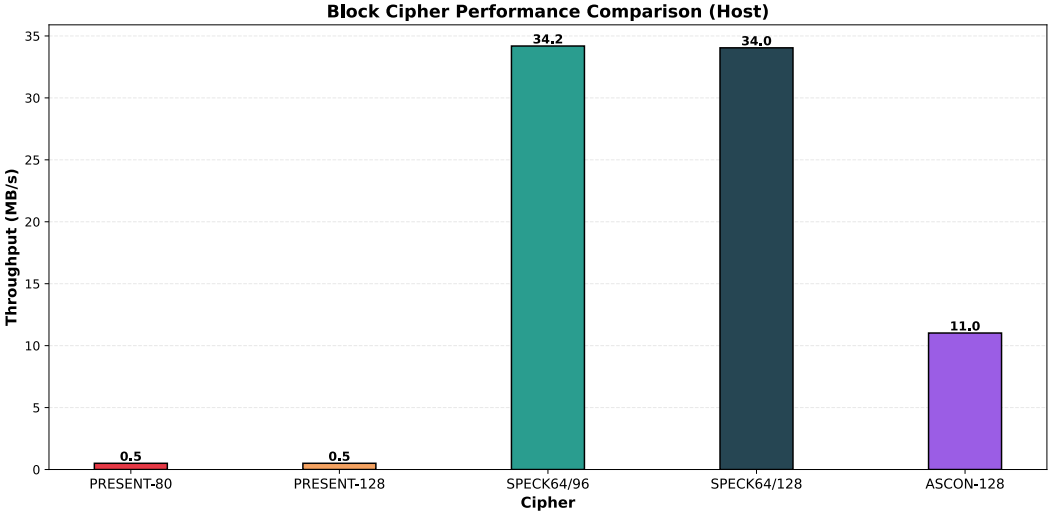
Benchmarking Approach

- Hardware timer for cycle-accurate measurements
- 1000+ iterations for statistical accuracy
- Warm-up runs before measurements
- Test vector validation for correctness

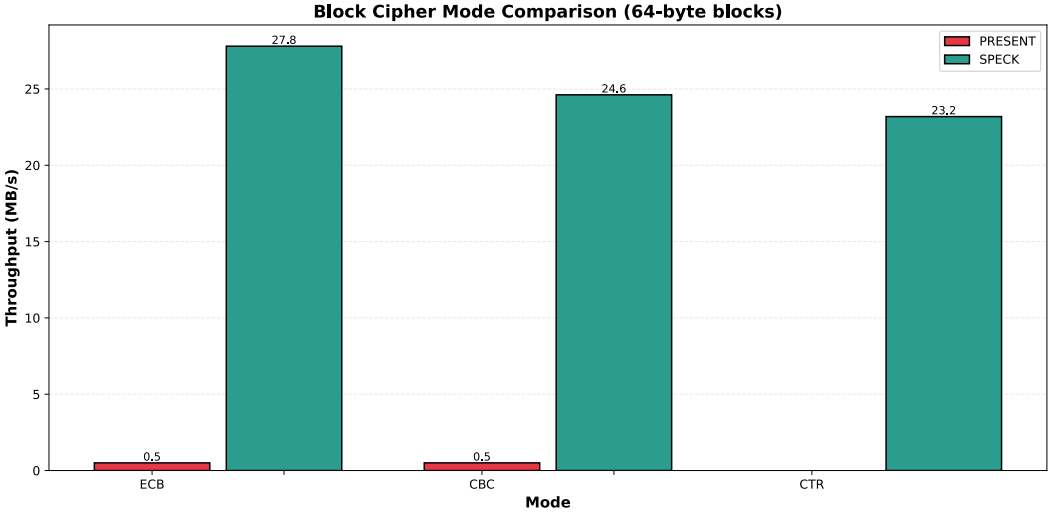
Benchmarks

Algorithm	Platform	ns/block	CPB	Mbps
PRESENT-80	Host	16,949	2,119	0.47
PRESENT-80	Pico	23,665	2,958	0.34
PRESENT-128	Host	16,990	2,124	0.47
PRESENT-128	Pico	24,542	3,068	0.33
SPECK-64/96	Host	246	31	32.52
SPECK-64/96	Pico	336	42	23.81
SPECK-64/128	Host	245	31	32.65
SPECK-64/128	Pico	338	42	23.67
ASCON-128	Host	1,518	95	10.54
ASCON-128	Pico	2,117	132	7.56

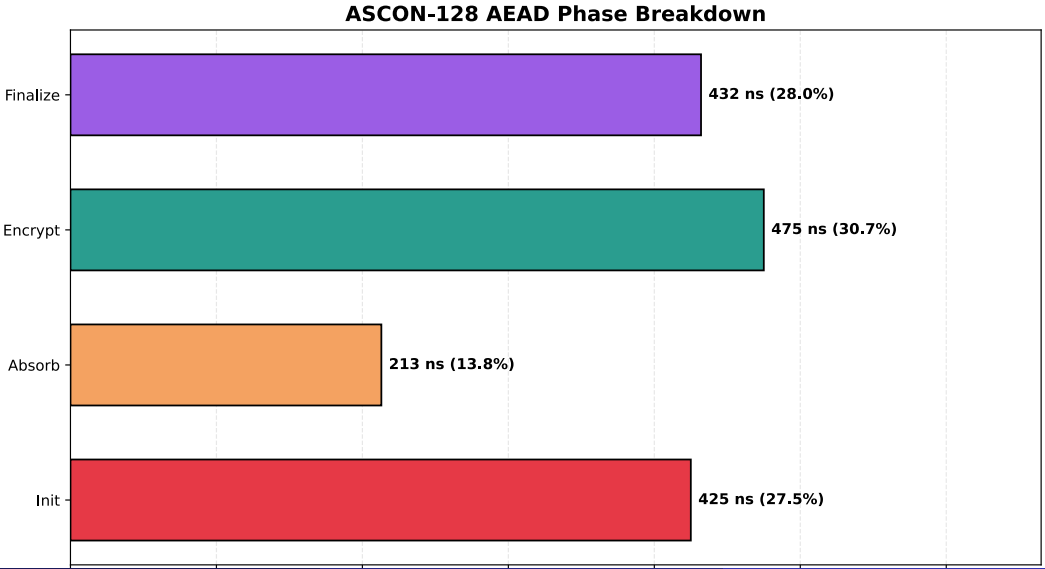
Visual Results: Block Cipher Comparison



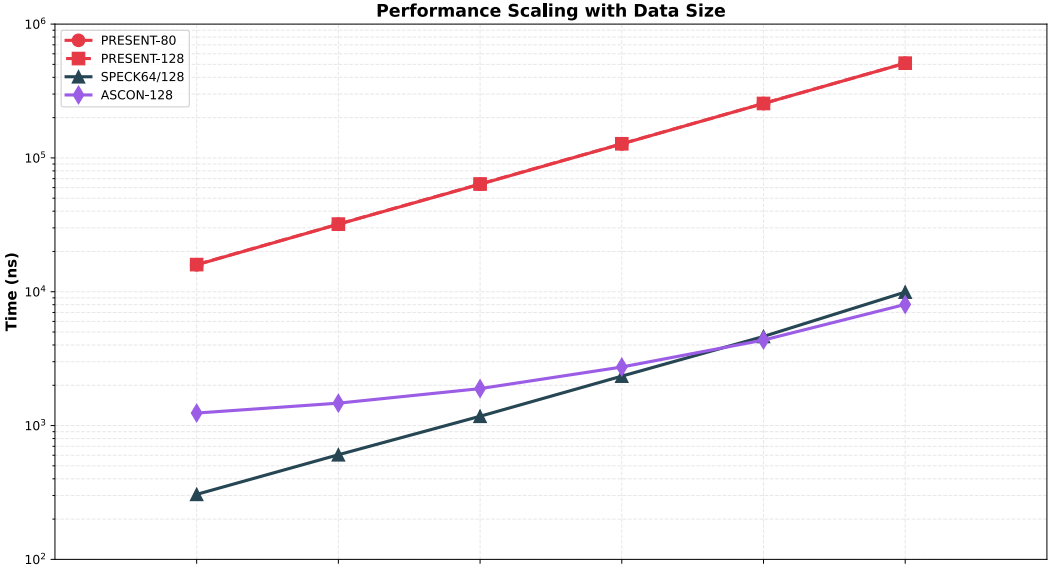
Visual Results: Mode Comparison



Visual Results: ASCON Phase Breakdown



Visual Results: Scaling Analysis



Analysis

Performance Observations

- **SPECK** is the fastest: 23.67 Mbps on Pico, approximately 70x faster than PRESENT
- **ASCON** provides good balance with built-in authentication (7.56 Mbps on Pico)
- **PRESENT** is the slowest on software but designed for hardware efficiency
- CTR mode offers best performance among encryption modes for SPECK
- ASCON's initialization overhead is amortized over larger message sizes

Memory Analysis

- **SPECK** has the smallest ROM footprint due to ARX-only operations (no lookup tables)
- **PRESENT** requires additional storage for S-box tables but remains compact
- **ASCON** has the largest code size due to 320-bit state and complex permutations

Conclusion and Next Steps

Conclusion

- PRESENT, SPECK, and ASCON were evaluated using Rust-based implementations on Raspberry Pi Pico
- SPECK provides superior performance, PRESENT offers minimal memory footprint
- ASCON delivers strong modern security with functional completeness (NIST Standard 2023)

Short-Term Next Steps

- NIST KAT Tests: Integrate official Known Answer Test vectors
- Code Size Analysis: Static analysis for .text/.data/.bss sections
- Statistical Rigor: Report median/min/max across multiple runs

Medium-Term Next Steps

- FELICS Comparison: Compare against established lightweight crypto benchmarks
- SUPERCOP-Style Timing: Implement cycle-accurate benchmarking
- Side-Channel Discussion: Analyze timing attack resistance for ARX ciphers

Thank You